

torch runtimes backing the Docling pipeline. We will deliver updates on this topic at in a future version of this report.

Table 1: Runtime characteristics of Docling with the standard model pipeline and settings, on our test dataset of 225 pages, on two different systems. OCR is disabled. We show the time-to-solution (TTS), computed throughput in pages per second, and the peak memory used (resident set size) for both the Docling-native PDF backend and for the pypdfium backend, using 4 and 16 threads.

CPU	Thread budget	native backend			pypdfium backend		
		TTS	Pages/s	Mem	TTS	Pages/s	Mem
Apple M3 Max (16 cores)	4	177 s	1.27	6.20 GB	103 s	2.18	2.56 GB
	16	167 s	1.34		92 s	2.45	
Intel(R) Xeon E5-2690 (16 cores)	4	375 s	0.60	6.16 GB	239 s	0.94	2.42 GB
	16	244 s	0.92		143 s	1.57	

5 Applications

Thanks to the high-quality, richly structured document conversion achieved by Docling, its output qualifies for numerous downstream applications. For example, Docling can provide a base for detailed enterprise document search, passage retrieval or classification use-cases, or support knowledge extraction pipelines, allowing specific treatment of different structures in the document, such as tables, figures, section structure or references. For popular generative AI application patterns, such as retrieval-augmented generation (RAG), we provide *quackling*, an open-source package which capitalizes on Docling’s feature-rich document output to enable document-native optimized vector embedding and chunking. It plugs in seamlessly with LLM frameworks such as LlamaIndex [8]. Since Docling is fast, stable and cheap to run, it also makes for an excellent choice to build document-derived datasets. With its powerful table structure recognition, it provides significant benefit to automated knowledge-base construction [11, 10]. Docling is also integrated within the open IBM data prep kit [6], which implements scalable data transforms to build large-scale multi-modal training datasets.

6 Future work and contributions

Docling is designed to allow easy extension of the model library and pipelines. In the future, we plan to extend Docling with several more models, such as a figure-classifier model, an equation-recognition model, a code-recognition model and more. This will help improve the quality of conversion for specific types of content, as well as augment extracted document metadata with additional information. Further investment into testing and optimizing GPU acceleration as well as improving the Docling-native PDF backend are on our roadmap, too.

We encourage everyone to propose or implement additional features and models, and will gladly take your inputs and contributions under review. The codebase of Docling is open for use and contribution, under the MIT license agreement and in alignment with our contributing guidelines included in the Docling repository. If you use Docling in your projects, please consider citing this technical report.

References

- [1] J. AI. Easyocr: Ready-to-use ocr with 80+ supported languages. <https://github.com/JaidedAI/EasyOCR>, 2024. Version: 1.7.0.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala. Pytorch 2: Faster

Table 2: Prediction performance (mAP@0.5-0.95) of object detection networks on DocLayNet test set. The MRCNN (Mask R-CNN) and FRCNN (Faster R-CNN) models with ResNet-50 or ResNet-101 backbone were trained based on the network architectures from the *detrone2* model zoo (Mask R-CNN R50, R101-FPN 3x, Faster R-CNN R101-FPN 3x), with default configurations. The YOLO implementation utilized was YOLOv5s [13]. All models were initialised using pre-trained weights from the COCO 2017 dataset.

	human	MRCNN R50	FRCNN R101	YOLO v5s6
Caption	84-89	68.4	71.5	77.7
Footnote	83-91	70.9	71.8	77.2
Formula	83-85	60.1	63.4	66.2
List-item	87-88	81.2	80.8	81.0
Page-footer	93-94	61.6	59.3	61.1
Page-header	85-89	71.9	70.0	72.0
Picture	69-71	71.7	72.7	72.0
Section-header	83-84	67.6	69.3	68.4
Table	77-81	82.2	82.9	82.2
Text	84-86	84.6	85.8	85.4
Title	60-72	76.7	80.4	79.9
All	82-83	72.4	73.5	76.8

to avoid this at any cost in order to have clear, unbiased baseline numbers for human document-layout annotation. Third, we introduced the feature of *snapping* boxes around text segments to obtain a pixel-accurate annotation and again reduce time and effort. The CCS annotation tool automatically shrinks every user-drawn box to the minimum bounding-box around the enclosed text-cells for all purely text-based segments, which excludes only *Table* and *Picture*. For the latter, we instructed annotation staff to minimise inclusion of surrounding whitespace while including all graphical lines. A downside of snapping boxes to enclosed text cells is that some wrongly parsed PDF pages cannot be annotated correctly and need to be skipped. Fourth, we established a way to flag pages as *rejected* for cases where no valid annotation according to the label guidelines could be achieved. Example cases for this would be PDF pages that render incorrectly or contain layouts that are impossible to capture with non-overlapping rectangles. Such rejected pages are not contained in the final dataset. With all these measures in place, experienced annotation staff managed to annotate a single page in a typical timeframe of 20s to 60s, depending on its complexity.

5 EXPERIMENTS

The primary goal of DocLayNet is to obtain high-quality ML models capable of accurate document-layout analysis on a wide variety of challenging layouts. As discussed in Section 2, object detection models are currently the easiest to use, due to the standardisation of ground-truth data in COCO format [16] and the availability of general frameworks such as *detrone2* [17]. Furthermore, baseline numbers in PubLayNet and DocLayNet were obtained using standard object detection models such as Mask R-CNN and Faster R-CNN. As such, we will relate to these object detection methods in this

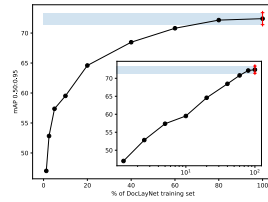


Figure 3: Prediction performance (mAP@0.5-0.95) of a Mask R-CNN network with ResNet50 backbone trained on increasing fractions of the DocLayNet dataset. The learning curve flattens around the 80% mark, indicating that increasing the size of the DocLayNet dataset with similar data will not yield significantly better predictions.

paper and leave the detailed evaluation of more recent methods mentioned in Section 2 for future work.

In this section, we will present several aspects related to the performance of object detection models on DocLayNet. Similarly as in PubLayNet, we will evaluate the quality of their predictions using mean average precision (mAP) with 10 overlaps that range from 0.5 to 0.95 in steps of 0.05 (mAP@0.5-0.95). These scores are computed by leveraging the evaluation code provided by the COCO API [16].

Baselines for Object Detection

In Table 2, we present baseline experiments (given in mAP) on Mask R-CNN [12], Faster R-CNN [11], and YOLOv5 [13]. Both training and evaluation were performed on RGB images with dimensions of 1025 × 1025 pixels. For training, we only used one annotation in case of redundantly annotated pages. As one can observe, the variation in mAP between the models is rather low, but overall between 6 and 10% lower than the mAP computed from the pairwise human annotations on triple-annotated pages. This gives a good indication that the DocLayNet dataset poses a worthwhile challenge for the research community to close the gap between human recognition and ML approaches. It is interesting to see that Mask R-CNN and Faster R-CNN produce very comparable mAP scores, indicating that pixel-based image segmentation derived from bounding-boxes does not help to obtain better predictions. On the other hand, the more recent YOLOv5x model does very well and even outperforms humans on selected labels such as *Text*, *Table* and *Picture*. This is not entirely surprising, as *Text*, *Table* and *Picture* are abundant and the most visually distinctive in a document.

Table 2: Prediction performance (mAP@0.5-0.95) of object detection networks on DocLayNet test set. The MRCNN (Mask R-CNN) and FRCNN (Faster R-CNN) models with ResNet-50 or ResNet-101 backbone were trained based on the network architectures from the *detrone2* model zoo (Mask R-CNN R50, R101-FPN 3x, Faster R-CNN R101-FPN 3x), with default configurations. The YOLO implementation utilized was YOLOv5s [13]. All models were initialised using pre-trained weights from the COCO 2017 dataset.

	human	MRCNN R50	FRCNN R101	YOLO v5s6
Caption	84-89	68.4	71.5	77.7
Footnote	83-91	70.9	71.8	77.2
Formula	83-85	60.1	63.4	66.2
List-item	87-88	81.2	80.8	81.0
Page-footer	93-94	61.6	59.3	61.1
Page-header	85-89	71.9	70.0	72.0
Picture	69-71	71.7	72.7	72.0
Section-header	83-84	67.6	69.3	68.4
Table	77-81	82.2	82.9	82.2
Text	84-86	84.6	85.8	85.4
Title	60-72	76.7	80.4	79.9
All	82-83	72.4	73.5	76.8

to avoid this at any cost in order to have clear, unbiased baseline numbers for human document-layout annotation. Third, we introduced the feature of *snapping* boxes around text segments to obtain a pixel-accurate annotation and again reduce time and effort. The CCS annotation tool automatically shrinks every user-drawn box to the minimum bounding-box around the enclosed text-cells for all purely text-based segments, which excludes only *Table* and *Picture*. For the latter, we instructed annotation staff to minimise inclusion of surrounding whitespace while including all graphical lines. A downside of snapping boxes to enclosed text cells is that some wrongly parsed PDF pages cannot be annotated correctly and need to be skipped. Fourth, we established a way to flag pages as *rejected* for cases where no valid annotation according to the label guidelines could be achieved. Example cases for this would be PDF pages that render incorrectly or contain layouts that are impossible to capture with non-overlapping rectangles. Such rejected pages are not contained in the final dataset. With all these measures in place, experienced annotation staff managed to annotate a single page in a typical timeframe of 20s to 60s, depending on its complexity.

5 EXPERIMENTS

The primary goal of DocLayNet is to obtain high-quality ML models capable of accurate document-layout analysis on a wide variety of challenging layouts. As discussed in Section 2, object detection models are currently the easiest to use, due to the standardisation of ground-truth data in COCO format [16] and the availability of general frameworks such as *detrone2* [17]. Furthermore, baseline numbers in PubLayNet and DocLayNet were obtained using standard object detection models such as Mask R-CNN and Faster R-CNN. As such, we will relate to these object detection methods in this

Figure 3: Prediction performance (mAP@0.5-0.95) of a Mask R-CNN network with ResNet50 backbone trained on increasing fractions of the DocLayNet dataset. The learning curve flattens around the 80% mark, indicating that increasing the size of the DocLayNet dataset with similar data will not yield significantly better predictions.

paper and leave the detailed evaluation of more recent methods mentioned in Section 2 for future work.

In this section, we will present several aspects related to the performance of object detection models on DocLayNet. Similarly as in PubLayNet, we will evaluate the quality of their predictions using mean average precision (mAP) with 10 overlaps that range from 0.5 to 0.95 in steps of 0.05 (mAP@0.5-0.95). These scores are computed by leveraging the evaluation code provided by the COCO API [16].

Baselines for Object Detection

In Table 2, we present baseline experiments (given in mAP) on Mask R-CNN [12], Faster R-CNN [11], and YOLOv5 [13]. Both training and evaluation were performed on RGB images with dimensions of 1025 × 1025 pixels. For training, we only used one annotation in case of redundantly annotated pages. As one can observe, the variation in mAP between the models is rather low, but overall between 6 and 10% lower than the mAP computed from the pairwise human annotations on triple-annotated pages. This gives a good indication that the DocLayNet dataset poses a worthwhile challenge for the research community to close the gap between human recognition and ML approaches. It is interesting to see that Mask R-CNN and Faster R-CNN produce very comparable mAP scores, indicating that pixel-based image segmentation derived from bounding-boxes does not help to obtain better predictions. On the other hand, the more recent YOLOv5x model does very well and even outperforms humans on selected labels such as *Text*, *Table* and *Picture*. This is not entirely surprising, as *Text*, *Table* and *Picture* are abundant and the most visually distinctive in a document.

Figure 3: Page 6 of the DocLayNet paper. If recognized, metadata such as authors are appearing first under the title. Elements recognized as page headers or footers are suppressed in Markdown to deliver uninterrupted content in reading order. Tables are inserted in reading order. The paragraph in "5. Experiments" wrapping over the column end is broken up in two and interrupted by the table.